

Generally moving objects using pointers in unmanaged languages (like C) is **hard**

TODO: Move the object located at **0xDEADBEEF** somewhere else

The runtime sees an **untyped, unstructured** array of bits

Which bits are pointers? Which are not?

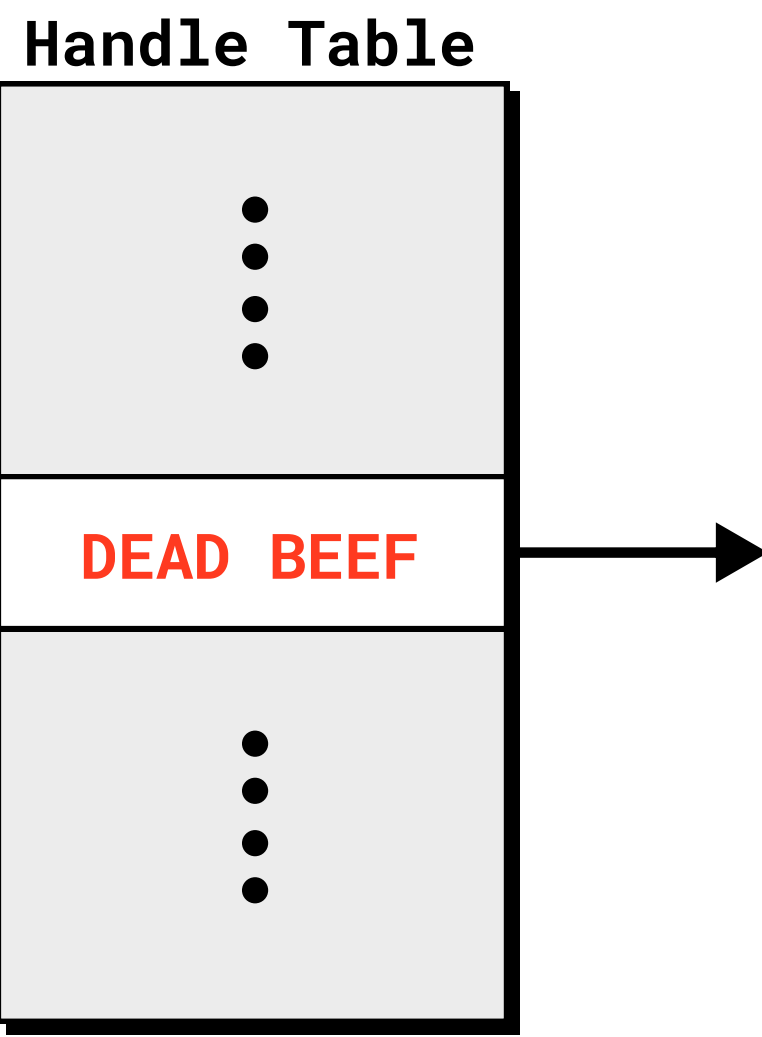
Is **this** a pointer to our object?

```
271f 3145 b236 fbfb df79 6379 fc8d b953
ee05 ebb6 b5c6 9233 28a1 5a6b 6fa6 82a2
6834 c565 21fa 970d 834e f70d 868b cd7f
cc52 e0bd 3152 210d da75 d0c4 c1a8 6725
ef19 DEAD BEEF a736 ce0e 4d31 acd2 c19e
12c8 e852 1def d7ff 80dd 95f4 bf04 221e
8306 dc8f 9d85 7a7d 59b4 62c3 84f5 8ebe
e792 b73d fec5 438a 6e08 bbf9 77bd 1964
52a6 fb9f 64d5 d1e9 9713 ae08 bfce 1b80
276f e679 63f1 869b c7c5 1c7c dc82 b265
faa3 9d35 9f64 8f3e 3119 b6f0 c8b5 5372
f5ba 013d 305a bd7d 99cd cc0a 65db 3d85
ee2f 0241 6e93 DEAD BEEF d940 be71 2957
5b6c 890c 0267 79d6 425f 41d5 b6a6 eb60
6799 b523 79c1 09f4 05a5 c503 c1b2 305f
a6b3 DEAD BEEF d9ff 979e 7505 0263 d62b
afc0 0e92 9640 e5fc d2dc ba54 e7b2 3300
4bb9 2404 e690 15ed 763e 206f 03ad 4602
dad1 61a7 8c48 c734 3a19 bb1d 603d 9ca8
9d9a d59a f81e 26db 52cd 3573 3c7e b31b
5a97 9f46 c4f1 ac4a fa21 DEAD BFF0 f3ad
3b81 c81e e906 a5a2 a1c8 c8ee a009 a049
```

Moving objects using handles in unmanaged languages is **trivial**

TODO: Move the object located at **0x80000000** somewhere else

```
271f 3145 b236 fbfb df79 6379 fc8d b953
ee05 ebb6 b5c6 9233 28a1 5a6b 6fa6 82a2
6834 c565 21fa 970d 834e f70d 868b cd7f
cc52 e0bd 3152 210d da75 d0c4 c1a8 6725
ef19 8000 0000 a736 ce0e 4d31 acd2 c19e
12c8 e852 1def d7ff 80dd 95f4 bf04 221e
8306 dc8f 9d85 7a7d 59b4 62c3 84f5 8ebe
e792 b73d fec5 438a 6e08 bbf9 77bd 1964
52a6 fb9f 64d5 d1e9 9713 ae08 bfce 1b80
276f e679 63f1 869b c7c5 1c7c dc82 b265
faa3 9d35 9f64 8f3e 3119 b6f0 c8b5 5372
f5ba 013d 305a bd7d 99cd cc0a 65db 3d85
ee2f 0241 6e93 8000 0000 d940 be71 2957
5b6c 890c 0267 79d6 425f 41d5 b6a6 eb60
6799 b523 79c1 09f4 05a5 c503 c1b2 305f
a6b3 DEAD BEEF d9ff 979e 7505 0263 d62b
afc0 0e92 9640 e5fc d2dc ba54 e7b2 3300
4bb9 2404 e690 15ed 763e 206f 03ad 4602
dad1 61a7 8c48 c734 3a19 bb1d 603d 9ca8
9d9a d59a f81e 26db 52cd 3573 3c7e b31b
5a97 9f46 c4f1 ac4a fa21 8000 0101 f3ad
3b81 c81e e906 a5a2 a1c8 c8ee a009 a049
```



With **handles**, only one **pointer** needs to be patched

Unmanaged languages have **UNTYPED MEMORY**

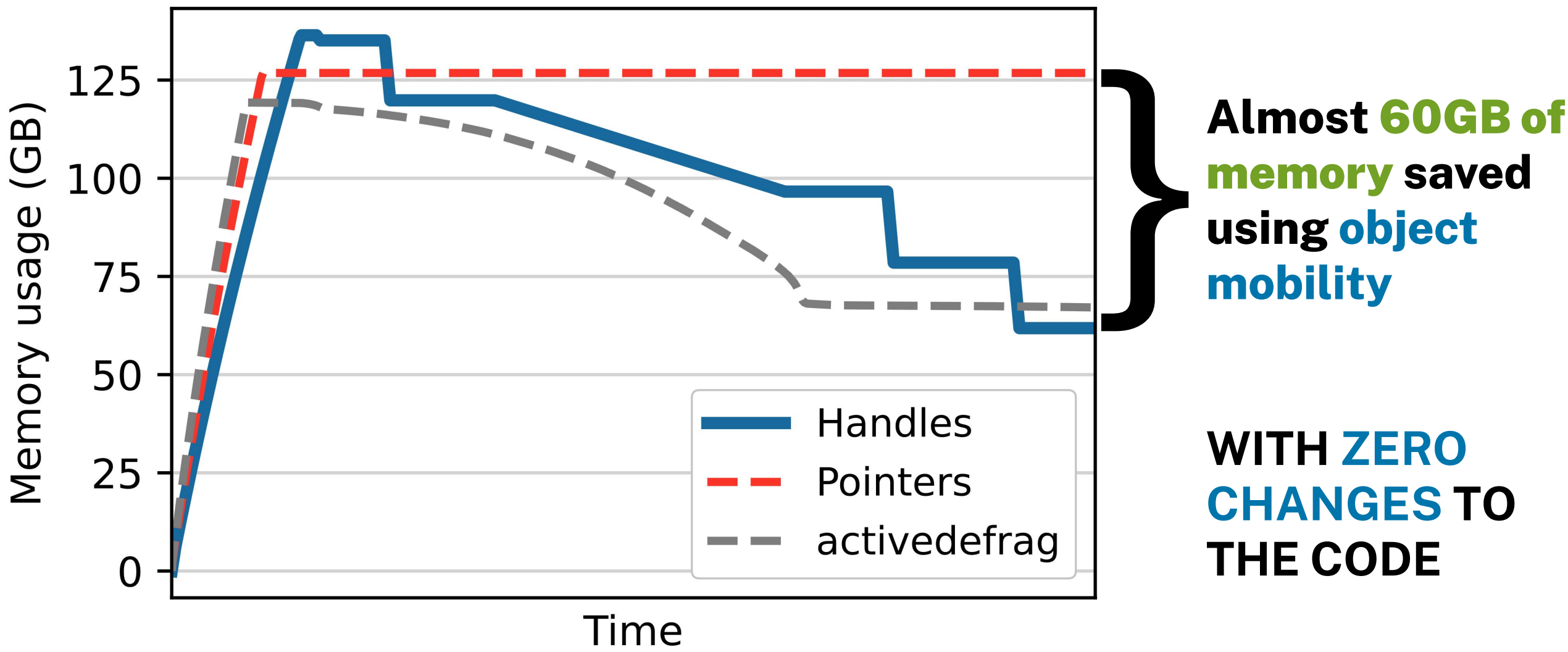
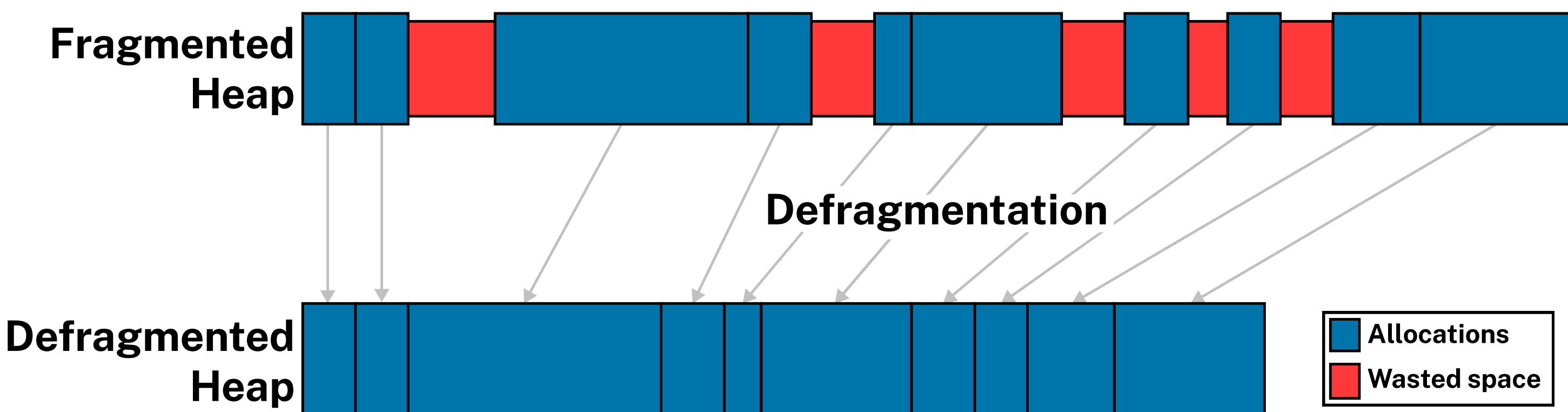
- Cannot distinguish between pointers and integers.
- To move an object, you need to find all pointers and update them.
- What if the dev uses something like an XOR list or pointer tagging?

Runtime systems in these languages miss out on...

- Memory Defragmentation
- Generational Memory Management
- Locality-aware rearrangement
- Object-granularity swapping / Compression

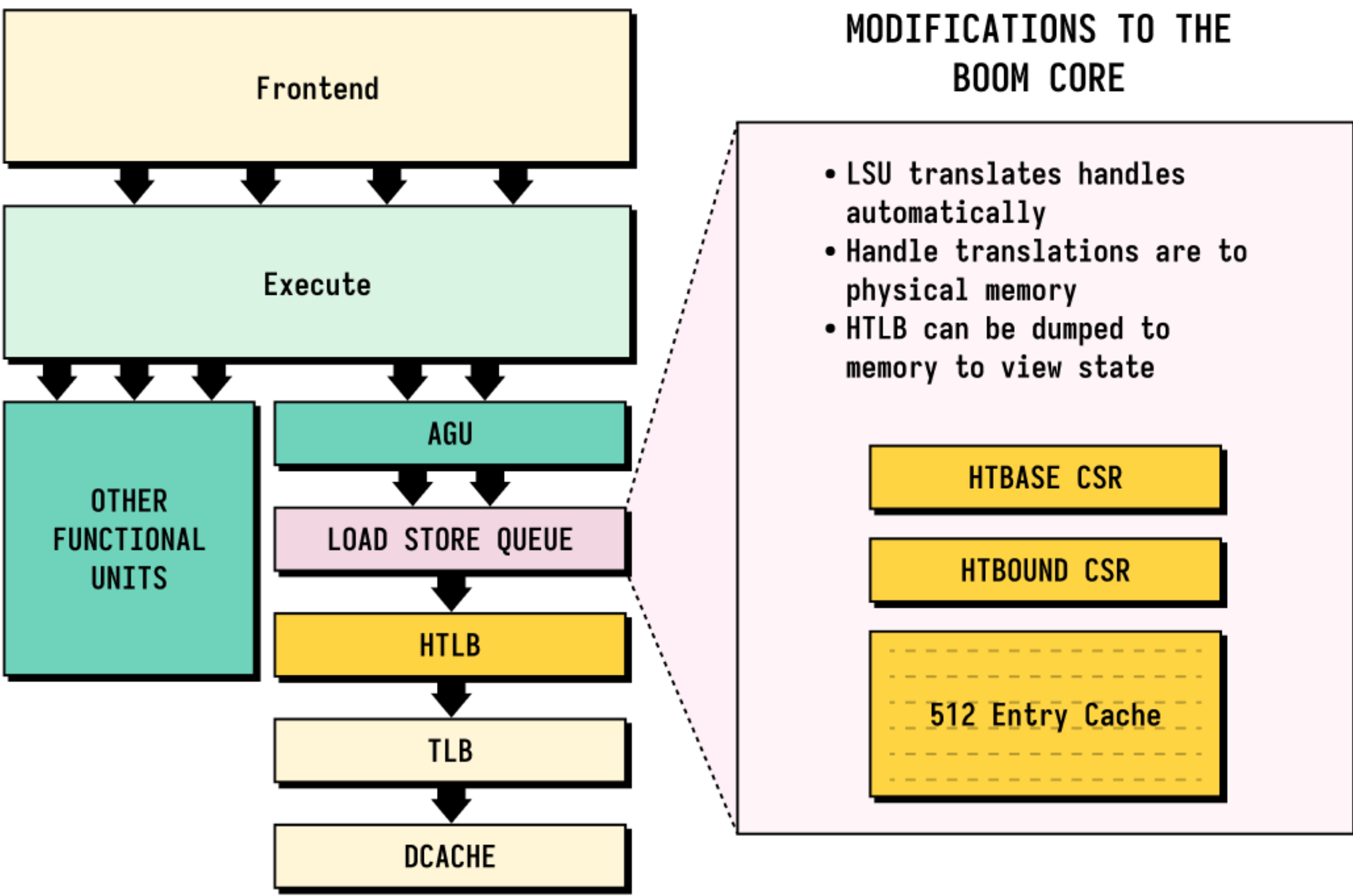
Can we **transparently** move heap objects in **unmanaged languages**?

Handles Enable Heap Defragmentation, **Saving Considerable Memory** (ASPLOS'24)



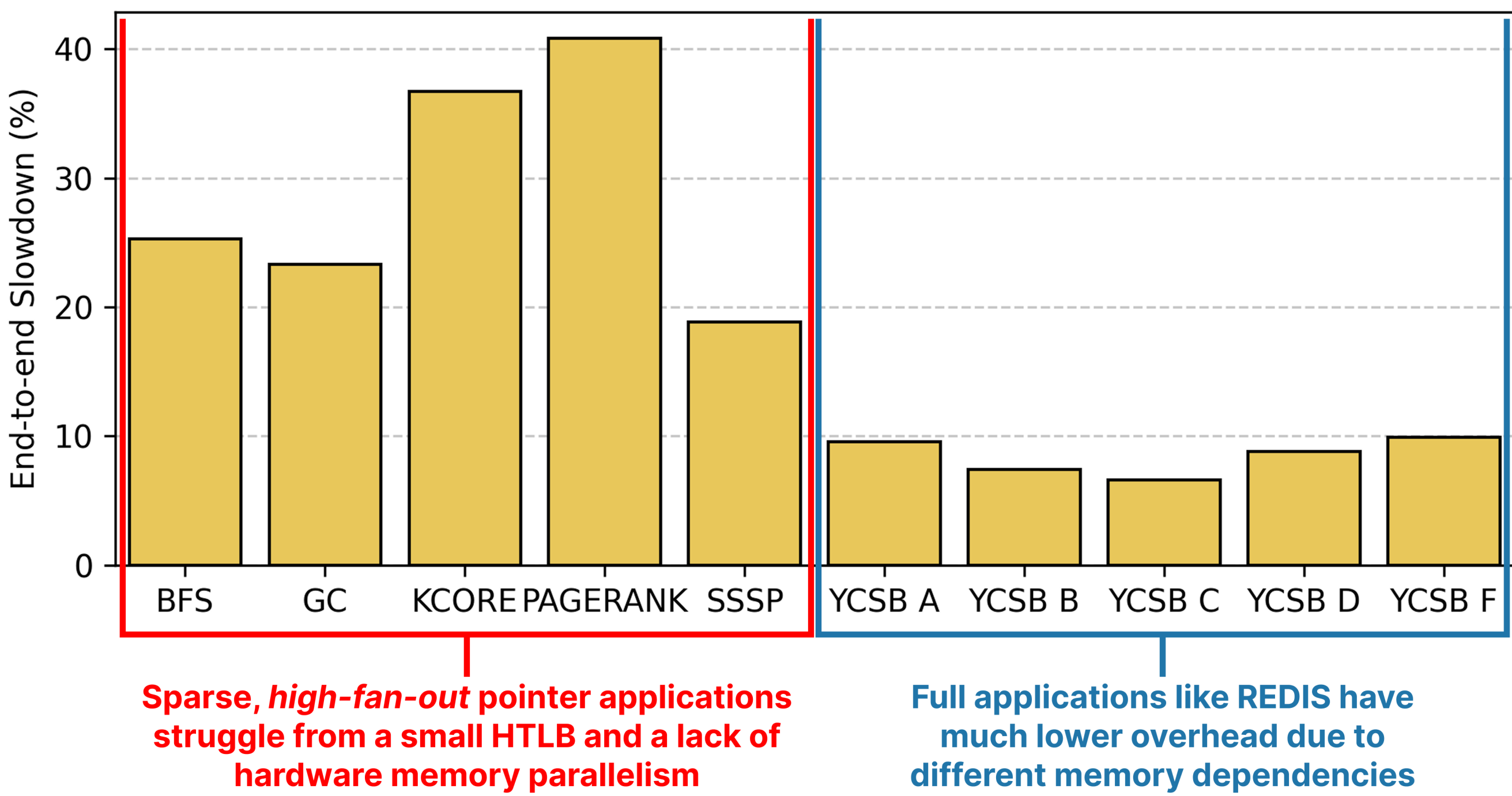
Yukon: Hardware for **Ubiquitous Handles**

- Previously, a compiler change was required to use Handles
- What about things like JIT compilers, assembly, and code we don't have the source for?
- What hardware would be needed to make this happen?



Initial overhead is competitive, bounded by BOOM's memory system

- Initial implementation in RISC-V BOOM has mixed results.
- Expected slowdown from additional serializing loads.
- BOOM memory system has **no memory parallelism**!



Next Steps:

- Evaluate hardware vs. software handles
 - Capabilities such as handle faults, access tracing, and more.
 - Gem5 simulation** to test performance on a better memory system?
 - Virtual memory with information at the granularity of individual objects
- All applications, without changes, can now use handles entirely transparently!**
 - `malloc` just returns handles now, enabling endless runtime techniques in previously impossible places